

Project 4: Raft Log Consensus and Persistence

Introduction

In this lab, you'll extend your Raft draft to keep a consistent and replicated log of operations.

The leader begins the process of adding a new operation to the log through the `Start()` call.

The leader then sends new operations to the other servers using `AppendEntries` RPCs.

This project builds on Project 3 by adding log agreement, persistent Raft state, and testing against a key/value service.

Your Work

Over the course of the following exercises, you will have to write or modify the following files yourself. Do not modify other files.

- `raft.go`

Provided Code

The structure of the project should look like:

```
|-- go.mod
|-- kvraft
|   |-- client.go
|   |-- common.go
|   |-- config.go
|   |-- server.go
|   |-- test_test.go
|-- labgob
|   |-- labgob.go
|   |-- test_test.go
|-- labrpc
|   |-- labrpc.go
|   |-- test_test.go
|-- linearizability
|   |-- bitset.go
|   |-- linearizability.go
|   |-- model.go
|   |-- models.go
|-- raft
    |-- config.go
```

```
|-- persister.go
|-- raft.go <- REPLACE WITH YOUR OWN raft.go
|-- test_test.go
`-- util.go
```

Project 4 Requirements

This project has three parts:

- Project 4.A: Log consensus
- Project 4.B: Persistency
- Project 4.C: Test your Raft with KVraft

Project 4.A: Log Consensus

It is necessary that Raft maintains a consistent, replicated log of operations.

In this part, you will implement the leader and follower code to append new log entries.

You will implement `Start()`, complete the `AppendEntries` RPC structs, send them, fill out the `AppendEntry` RPC handler, and advance the `commitIndex` of the leader.

You should first pass the `TestBasicAgree4A()` test in `test_test.go`.

Then you should get all the 4A tests to pass:

```
go test -run 4A ./raft
```

The 4A tests will be used to grade this assignment.

You must also ensure that your code is thread-safe. You can check this using the `-race` parameter:

```
go test -race -run 4A ./raft
```

Project 4.B: Persistency

If a Raft-based server reboots, it should resume service where it left off.

Therefore, it is necessary that Raft keep persistent state that survives a reboot.

This is mentioned in Figure 2 of the paper, and `raft.go` contains examples of how to save and restore persistent state.

In the real world, this would be done by writing Raft's persistent state to disk each time it changes and reading from the disk after a reboot.

For this lab, you will instead use a `Persister` object in `persister.go`.

Whoever calls `Raft.Make()` provides a `Persister` that initially holds Raft's most recently persisted state, if there is any.

Raft should initialize its state from the `Persister` and then use it to save its persistent state each time the state changes.

For this, use the `Persister`'s `ReadRaftState()` and `SaveRaftState()` methods.

In this part, you will complete the functions `persist()` and `readPersist()` in `raft.go` by adding code to save and restore persistent state.

You will have to encode, or serialize, the array state as a byte array in order to pass it to the `Persister`.

To do this, use the `labgob` encoder that we provide. See comments in `persist()` and `readPersist()`.

`labgob` is inspired by Go's `gob` encoder; the only difference is that `labgob` checks if your encoding structures have lower-case field names.

You must determine where in the Raft protocol your servers are required to persist their state and insert calls to `persist()` at those points.

In `Raft.Make()` there is already a call to `readPersist()`.

Once this is done, you should pass the remaining tests.

You may want to first pass the basic persistence test:

```
go test -run TestPersistA4B ./raft
```

Then tackle the remaining 4B tests:

```
go test -run 4B ./raft
```

Note that many of the 4B tests involve servers failing and the network losing RPC requests or replies.

Project 4.C: Test Your Raft with KVraft

So far, you have built a Raft server.

Now it is time to use your server.

In this part, you will make your Raft cooperate with a key/value service.

A simple KV service based on our Raft structure and its test cases are given.

Each of the key/value servers, or `kvserver`, has an associated Raft peer.

Clerks send `Put()`, `Append()`, and `Get()` RPCs to the `kvserver` whose associated Raft is the leader.

The `kvserver` code submits the `Put`, `Append`, or `Get` operation to Raft, so that the Raft log holds a sequence of `Put`, `Append`, and `Get` operations.

All of the `kvserver`s execute operations from the Raft log in order, applying the operations to their key/value databases.

The intent is for the servers to maintain identical replicas of the key/value database.

A Clerk sometimes does not know which `kvserver` is the Raft leader.

If the Clerk sends an RPC to the wrong `kvserver`, or if it cannot reach a `kvserver`, the Clerk retries by sending to a different `kvserver`.

If the key/value service commits the operation to its Raft log and applies the operation to the key/value state machine, the leader reports the result to the Clerk by responding to its RPC.

If the operation fails to commit, for example if the leader is replaced, the server reports an error, and the Clerk retries with a different server.

Your work is running test cases for KVraft and making sure they all pass.

KVraft test cases might catch semantic problems in your Raft server.

If the test set returns `ok`, all you should do is evaluate the test results.

Any failure means that your Raft is somehow incorrect, so you should debug your Raft to find the bug.

Test your code with:

```
go test ./kvraft
```

For most of you with a complete Raft, there would be few things to work on for this part.

Once a bug shows up, debugging can be challenging, so start early.

Hints and Good Ideas

- You must implement the election restriction detailed in Section 5.4.1 of the paper.
- Failing early 4A tests may stem from holding unneeded elections, such as an election even though the current leader is alive and communicating with all peers. This can prevent agreement in situations where the tester believes agreement is possible.
- Not sending out heartbeats immediately after winning an election or bugs in election timer management can cause unneeded elections.
- You may need to wait for certain events to occur within your code. Do this with Go's channels, Go's condition variables, or, as a last resort, by inserting a `time.Sleep(10 * time.Millisecond)` in each loop iteration.
- Allow for extra time to rewrite your implementation with insight gained from structuring concurrent code.

Submission Instructions

File contents: all files of the project.

1. Move to the parent directory of the project folder.

For example, if the project folder is located at `~/lab4_raft_adv`, run:

```
cd ..
```

2. Create the submission archive:

```
tar -zcvf lab4_raft_adv_<GROUP_NUMBER>.tgz lab4_raft_adv
```

3. Make sure to include all project files.

The TAs will run the following command to extract submissions:

```
tar -zxvf lab4_raft_adv_<GROUP_NUMBER>.tgz
```

Do not leave any code for printing/debugging.

Additional text output in the test result may significantly distract grading, so it will deduct minor points from your grade.

Do not include extra files unrelated to the project.